DOCUMENTAÇÃO TÉCNICA

WaveLens

Simulador de Cobertura Wi-Fi 2.5D e Diagnóstico de Campo Nativo

Versão 8.2 revisão 8.2.1

Atualizada em 18/10/2026

Stack: Flutter / Dart · Kotlin (Android) · GitHub Pages (PWA) · JS Bridge

Aplicativo: rogerdev5690.github.io/netfloor

Código: github.com/Rogerdev5690/netfloor · github.com/Rogerdev5690/netfloor-shell

Sumário

1. Visão Geral e Proposta do Produto 4
- 1.1 Conceito 4
- 1.2 Casos de uso 4
- 1.3 Módulos em resumo 4
- 1.4 Arquitetura Serverless / PWA 4
- 1.5 Nome do produto e rebranding (v8.2) 5

2. Arquitetura de Software e Stack Tecnológica 7
- 2.1 Stack 7
- 2.2 Repositórios e estrutura de pastas 7
- 2.3 Frontend Web/PWA (lib/) 8
- 2.4 Android Shell 9
- 2.5 Ponte de Comunicação (JS Bridge) 9
- 2.6 Ícone do aplicativo 11

3. Motor do Simulador e Engenharia de RF 12
- 3.1 Fórmula de propagação 12
- 3.2 Catálogo de hardware 12
- 3.3 Distância 3D e escala 12
- 3.4 Física de barreiras (ray-casting) 13
- 3.5 Arquitetura 2.5D multi-pavimento 13
- 3.6 Exemplos numéricos 14
- 3.7 Visualização do mapa de calor 14
- 3.8 Interação 15
- 3.9 Limitações do modelo 15
- 3.10 Simulação em 2.4 / 5 / 6 GHz 15

4. Módulo de Diagnóstico Mobile (Android Nativo) 17
- 4.1 Modos de operação 17
- 4.2 Requisitos e permissões do Android 17
- 4.3 Aba Espectro e Saúde do Canal 17
- 4.4 Aba Sinal (walk-through) 18
- 4.5 Aba Latência e Rede (ping duplo) 18
- 4.6 Controlador de diagnóstico 19
- 4.7 Modo simulação 19
- 4.8 Estado de validação 19

5. WaveLens Enterprise (v8.0) 20
- 5.1 Visão geral 20
- 5.2 Fluxo de trabalho típico em campo 20
- 5.3 Ferramentas do mapa e gestos 20
- 5.4 Modos de visualização 21
- 5.5 Pontos de medição e medição de campo 21
- 5.6 Persistência offline-first e arquivo .json 21
- 5.7 Laudo de vistoria em PDF 22
- 5.8 Assinatura digital do cliente 23
- 5.9 Entrega de arquivos (PDF e .json) 23
- 5.10 Estado de validação dos recursos da v8.0 23

6. Gestão de Plantas e Mídia 24
- 6.1 Modelo de dados 24
- 6.2 Cadeia de resolução da imagem 24
- 6.3 Biblioteca atual 24
- 6.4 Como adicionar uma nova planta 25
- 6.5 Upload em memória (Uint8List) 25
- 6.6 Política de conteúdo e licenças 25

6.7 Histórico de alterações 26

7. Guia de Instalação, Build e CI/CD 28

7.1 Ambiente de desenvolvimento (Windows 11) 28
7.2 Dependências (pubspec.yaml) 28
7.3 Permissões do Android (AndroidManifest.xml do Shell) 28
7.4 Desenvolvimento local 29
7.5 Build web de produção 29
7.6 Pipeline de implantação contínua (OTA) via GitHub Pages 29
7.7 Build e versionamento do APK Shell 30
7.8 Checklist de release 30
7.9 Solução de problemas 30
7.10 Segurança e privacidade 31

8. Limitações Conhecidas e Pendências 32

9. Manutenção desta Documentação (Regra de Projeto) 33

Apêndice A — Glossário 34

Apêndice B — Referência rápida de constantes 35

1. Visão Geral e Proposta do Produto

1.1 Conceito

O **WaveLens** é um aplicativo com duas frentes complementares:

1. **Simulador de Cobertura Wi-Fi.** O usuário escolhe uma planta baixa, posiciona roteadores de modelos reais (com potência de transmissão própria) e vê, em tempo real, um **mapa de calor** de sinal sobreposto à planta. O cálculo considera a distância em 3D, a atenuação por paredes (ray-casting) e a atenuação por lajes entre pavimentos.
2. **Diagnóstico de Campo Nativo (WaveLens Diagnostic).** Uma aba que, quando executada dentro do aplicativo Android (WaveLens Shell), lê os dados **reais** do aparelho: espectro de canais em 2.4 GHz e 5 GHz com saúde do canal, sinal (RSSI) em tempo real e latência (ping duplo: roteador local versus DNS da Internet), além da velocidade PHY negociada. Fora do Shell (navegador comum), a aba funciona em **modo simulação** com dados fictícios, claramente identificados.

A **versão 8.0 (Enterprise)** acrescenta recursos de campo e de relacionamento com o cliente: **laudo de vistoria em PDF** com assinatura digital, **calibração de escala por régua**, **materiais de parede** com perdas realistas, **simulação em 2.4 / 5 / 6 GHz**, **zoom e deslocamento** do mapa, **modos de visualização** (Apresentação e Diagnóstico) e **armazenamento offline** dos projetos com exportação/importação em `.json` (capítulo 5).

O nome da marca exibida ao usuário é **WaveLens** (o produto chamava-se **NetFloor** até a v8.1 — seção 1.5 explica o que muda e o que fica igual no rebranding).

1.2 Casos de uso

Perfil	Uso típico
Instalador / técnico de redes	Planejar onde posicionar roteadores ou APs antes da instalação; comparar modelos (Huawei, TP-Link Deco, UniFi).
Morador / pequeno escritório	Entender por que o Wi-Fi cai em certos cômodos; testar cobertura em sobrados e prédios com vários andares.
Diagnóstico em campo	Caminhar pelo ambiente observando o RSSI (walk-through), escolher o melhor canal e medir latência até o roteador e até a Internet.

1.3 Módulos em resumo

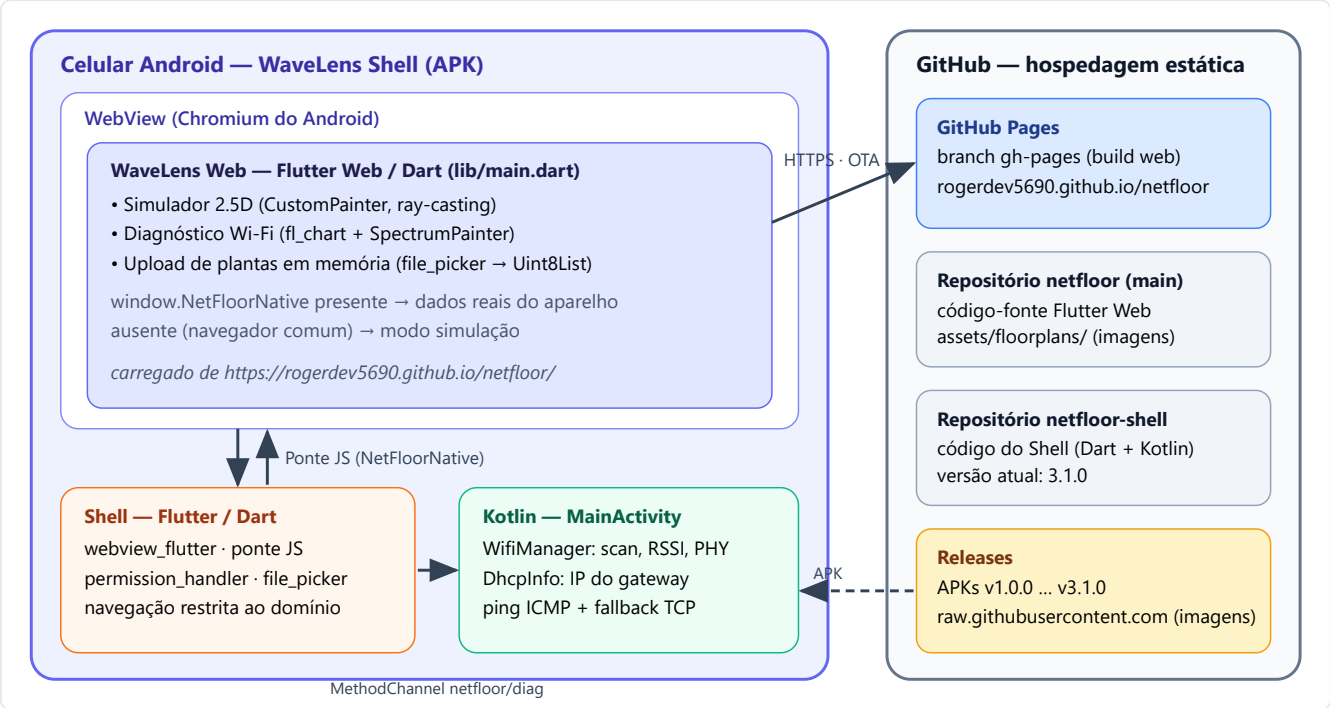
Módulo	Onde roda	Fonte dos dados
Simulador 2.5D (mapa de calor)	Navegador, PWA e Shell Android	Modelo matemático (seção 3)
Espectro de canais e saúde do canal	Shell Android (real); navegador (simulação)	`WifiManager.getScanResults()`
Monitor de sinal (walk-through)	Shell Android (real); navegador (simulação)	`WifiInfo.getRssi()` a cada 1 s
Latência e rede (ping duplo)	Shell Android (real); navegador (simulação)	`ping` do sistema; `DhcpInfo`
Biblioteca e upload de plantas	Todos os ambientes	Assets, rede (GitHub) e bytes em memória
Ferramentas de mapa (paredes, régua, pontos), banda e zoom	Todos os ambientes	Toque/mouse do usuário (seção 5.3)
Projetos offline e arquivo `.json`	Todos os ambientes	IndexedDB do navegador/WebView (seção 5.6)
Laudo de vistoria em PDF com assinatura	Todos os ambientes; salvar/compartilhar nativo no Shell 3.2+	Simulador + medições de campo (seções 5.7 a 5.9)

1.4 Arquitetura Serverless / PWA

O WaveLens **não possui servidor próprio, banco de dados ou API**. Todo o produto é composto de arquivos estáticos:

- O **aplicativo web** (Flutter Web compilado para JavaScript) é publicado em **GitHub Pages**, no endereço `https://rogerdev5690.github.io/netfloor/`.
- O **WaveLens Shell** é um APK Android que abre esse endereço dentro de uma `WebView` e acrescenta capacidades nativas (Wi-Fi, ping, seletor de arquivos) por meio de uma ponte JavaScript.
- Toda a lógica de negócio vive no navegador/WebView; nada é enviado a servidores de terceiros além do carregamento de arquivos estáticos do GitHub. Projetos, laudos e assinaturas ficam **no próprio aparelho** (IndexedDB) e só saem dele quando o usuário exporta ou compartilha um arquivo.

Atualização OTA (over-the-air). Como a interface vem do site, publicar uma nova versão do site atualiza o aplicativo instalado no celular na próxima abertura, sem reinstalar o APK. O APK só precisa ser reinstalado quando algo **nativo** muda (código Kotlin, permissões, versão do Shell). Para garantir isso, o build web usa `--pwa-strategy=none` (sem cache agressivo de service worker).



Arquitetura geral do WaveLens

1.5 Nome do produto e rebranding (v8.2)

Na v8.2 o produto passou a se chamar **WaveLens** (antes **NetFloor**). O rebranding trocou tudo o que o usuário vê — título do app, cabeçalhos, banners, metadados do laudo em PDF, nome do aplicativo Android (`android:label`) e ícone — mas **preservou deliberadamente** os identificadores técnicos internos, para não quebrar nada que já está publicado:

Identificador técnico	Continua sendo	Por quê
Repositórios GitHub	`Rogerdev5690/netfloor`, `Rogerdev5690/netfloor-shell`	Renomear o repositório mudaria a URL do site publicado (`rogerdev5690.github.io/netfloor/`) e todos os links já compartilhados
`applicationId` do Android	`com.netfloor.netfloor_shell`	Trocar o `applicationId` faz o Android tratar o app como **outro aplicativo**: quem já tem o app instalado passaria a ter dois ícones, em vez de receber uma atualização. Só o `android:label` (o nome exibido) mudou
Ponte JS `NetFloorNative`	`NetFloorNative` (`@JS('NetFloorNative')` no app web, `addJavaScriptChannel('NetFloorNative', ...)` no Shell)	É um contrato interno entre os dois apps (web e Shell); renomear exigiria publicar os dois ao mesmo tempo sem nenhuma janela de incompatibilidade

Identificador técnico	Continua sendo	Por quê
`MethodChannel('netfloor/diag')`	`netfloor/diag`	Mesmo motivo: contrato interno entre o Dart do Shell e o Kotlin
Formato dos arquivos `.json` exportados	`"format": "netfloor-project"`	Projetos já exportados por usuários continuam sendo aceitos na importação
Nomes de pacote Dart (`pubspec.yaml`)	`name: wavelens` / `name: wavelens_shell`	Atualizados na v8.2 (são só identificadores internos do Dart, sem risco: nenhum arquivo importa `package:netfloor/...`)

Em resumo: **tudo que aparece na tela mudou; os "canos" por trás continuam com o nome antigo.**

2. Arquitetura de Software e Stack Tecnológica

2.1 Stack

Camada	Tecnologia	Versão / observação	Função
Frontend Web/PWA	**Flutter / Dart**	Flutter 3.47.4 (stable), Dart 3.13.3	Interface, simulador, diagnóstico
Renderização 2D	`CustomPainter`, `Canvas`, `ImageFiltered`	SDK Flutter	Mapa de calor, ícones dos roteadores, espectro
Gráficos de linha	`fl_chart`	^1.2.0	Sinal (RSSI) e latência
Upload de arquivos	`file_picker`	^13.1.0	Plantas, logo e projetos `.json` em memória (`Uint8List`)
Laudo em PDF	`pdf` (100 % Dart)	^3.13.1	Geração do laudo no navegador/WebView
Imagens	`image`	^4.10.1	Codificação JPEG dos mapas de calor do laudo
Armazenamento local	`sembast` + `sembast_web`	^3.8.11 / ^2.4.6	Banco NoSQL sobre IndexedDB (offline-first)
Download de arquivos	`web` (`package:web`)	^1.1.1	`Blob` + âncora `download` no navegador
Fonte do PDF	Roboto (Apache 2.0)	`assets/fonts/`	Acentos e símbolos no laudo
Interop Web	`dart:js_interop`, `dart:js_interop_unsafe`	SDK Dart	Ponte com o Shell
Android Shell (UI)	**Flutter / Dart**	Shell 3.3.0 (build 6)	Hospeda a WebView e a ponte
WebView	`webview_flutter` + `webview_flutter_android`	^4.10.0 / ^4.14.1	Exibe o site e injeta `NetFloorNative`
Permissões	`permission_handler`	12.0.1	Localização e Nearby Wi-Fi
Android nativo	**Kotlin**	JDK 17 (Temurin 17.0.20)	`WifiManager`, ping, `DhcpInfo`, `MediaStore`, `FileProvider`
Build Android	Gradle + AGP (template Flutter)	SDK 36, build-tools 35/36	Empacotamento do APK
Hospedagem	**GitHub Pages**	branch `gh-pages`	Servir o app web
Distribuição do APK	**GitHub Releases**	`gh release create`	Baixar/instalar o Shell
Automação local	PowerShell, Bash, `gh` CLI, `git`	Windows 11	Build, deploy e versionamento

2.2 Repositórios e estrutura de pastas

Repositório	Conteúdo
`github.com/Rogerdev5690/netfloor` (branch `main`)	Código-fonte do app web, assets das plantas e esta documentação (`docs/`)
`github.com/Rogerdev5690/netfloor` (branch `gh-pages`)	Build web publicado (gerado, sobrescrito a cada deploy)
`github.com/Rogerdev5690/netfloor-shell` (branch `main`)	Código do Shell Android (Dart + Kotlin)
`github.com/Rogerdev5690/netfloor/releases`	APKs do Shell (v1.0.0 a v3.3.0)

Os nomes dos repositórios (`netfloor`, `netfloor-shell`) e a URL do site (`rogerdev5690.github.io/netfloor/`) permanecem como estavam antes do rebranding v8.2 — ver seção 1.5.

Estrutura local (`C:\Users\Roger\Desktop\Nova pasta\`):

```
netfloor/                    # app web (Flutter) — pubspec name: wavelens
  lib/main.dart              # importações + lista de módulos (part) + main()
  lib/app/ core/ simulador/  # 30 módulos "ETAPA n" (seção 2.3 e GUIA_DO_CODIGO.md)
  lib/diagnostico/ infra/ laudo/
  assets/floorplans/         # imagens das plantas
  assets/icon/               # icon.png (fonte dos ícones web/PWA, seção 2.6)
  assets/fonts/              # Roboto (Regular, Bold, Italic) usada no PDF
  pubspec.yaml
  run_web.bat                # servidor de desenvolvimento (porta 8090)
  docs/                      # esta documentação
    DOCUMENTACAO.md          # fonte da documentação
    build_pdf.py             # gerador do PDF
    diagramas/*.svg          # diagramas
    Documentacao_Tecnica_App.pdf
  android/                   # legado: build nativo da v1.0.0 (não usado)
netfloor_shell/              # Shell Android — pubspec name: wavelens_shell
  lib/main.dart              # WebView + ponte JS
  assets/icon/               # icon.png (fonte do flutter_launcher_icons, seção 2.6)
  android/app/src/main/
    AndroidManifest.xml      # permissões, android:label = "WaveLens"
    kotlin/com/netfloor/netfloor_shell/MainActivity.kt
```

2.3 Frontend Web/PWA (`lib/`)

Desde a revisão 8.2.1 o código do app web está dividido em **30 módulos** dentro de `lib/`. O `lib/main.dart` só tem as importações, a lista de módulos e a função `main()`. Os módulos usam `part` / `part of` do Dart: continuam sendo **uma única biblioteca** (mesmas importações, mesmos nomes privados `_`), apenas espalhada em arquivos pequenos. Cada arquivo começa com um cabeçalho **"ETAPA n"** (o que é, como funciona, usado por, onde mexer), pensado para explicar o app etapa por etapa.

Pasta	Etapas	Conteúdo	Classes principais
`app/`	1–3	Entrada do app, navegação inferior, nome/versão, temas	`WaveLensApp`, `WaveLensShell`, `kAppName`, `ViewMode`, `buildTheme`
`core/`	4–5	Utilitários e diálogos reutilizáveis	`_muted`, `_fmtDate`, `BusyDialog`, `confirmDialog`, `safeFileName`
`simulador/modelo`, `dados`, `estado`	6–10	Dados (roteadores, plantas, paredes, bandas, medições, laudo), biblioteca de plantas e o estado do projeto	`RouterModelSpec`, `FloorPlanDef`, `FloorState`, `WallSegment`, `RfBandSpec`, `ProjectDef`, `Measurement`, `ReportInfo`, `NetworkModel`
`simulador/fisica`	11	A física do sinal	`SignalField`, `computeCoverage`
`simulador/pintura`	12–16	Paleta de calor e todos os `CustomPainter`	`HeatmapPainter`, `FloorPlanPainter`, `ToolOverlayPainter`, `RadarPingPainter`, `PointMarkerPainter`, `RouterDevicePainter`
`simulador/tela`	17	A tela principal do simulador	`SimulatorPage`
`diagnostico/`	18–25	Ponte nativa, modelos, fontes de dados, controlador e as 3 abas	`NativeBridge`, `WifiNetwork`, `LinkInfo`, `DiagnosticsSource`, `DiagnosticsController`, `DiagnosticPage`, `SpectrumTab`, `SignalTab`, `LatencyTab`
`infra/`	26–27	Armazenamento local offline e entrega de arquivos	`AppStore`, `FileIO`
`laudo/`	28–30	Assinatura, tela do laudo e gerador do PDF	`SignaturePadPage`, `ReportPage`, `LaudoPdf`, `renderFloorRaster`

A tabela arquivo a arquivo, com o roteiro de explicação, está em `docs/GUIA_DO_CODIGO.md`. A divisão foi verificada mecanicamente: as 6.205 linhas de código não vazias dos módulos são idênticas às do antigo `main.dart` (diferença zero), o `flutter analyze` não mudou e o app foi testado depois da divisão.

Gerenciamento de estado. Usa-se `ChangeNotifier` com `AnimatedBuilder`, sem bibliotecas externas. `NetworkModel` guarda projeto, pavimentos (com escala e paredes), roteadores, pontos de medição, registros de diagnóstico e dados do laudo; `DiagnosticsController` guarda varredura, amostras de sinal e de ping. O `WaveLensShell` é dono dos dois e do `AppStore` (armazenamento), repassando-os às telas. Durante o arraste de um roteador só a camada do mapa de calor é redesenhada, mantendo a interação fluida.

Composição visual. A área do mapa fica dentro de um `InteractiveViewer` (zoom e deslocamento) e é um `Stack` de camadas: (1) imagem ou desenho da planta (com filtro de cor conforme o modo de visualização), (2) mapa de calor com desfoque, (3) paredes desenhadas e régua, (4) anéis pulsantes dos roteadores, (5) roteadores de outros andares (esmaecidos), pontos de medição e roteadores do andar, com tamanho constante na tela. Por cima, fora do zoom, ficam o selo de escala, a legenda e os botões de zoom.

Navegação. Uma `NavigationBar` alterna entre Simulador e Diagnóstico dentro de um `IndexedStack`, preservando o estado das duas telas. Cada tela tem seu `TickerMode`, de modo que animações e temporizadores da aba oculta ficam parados.

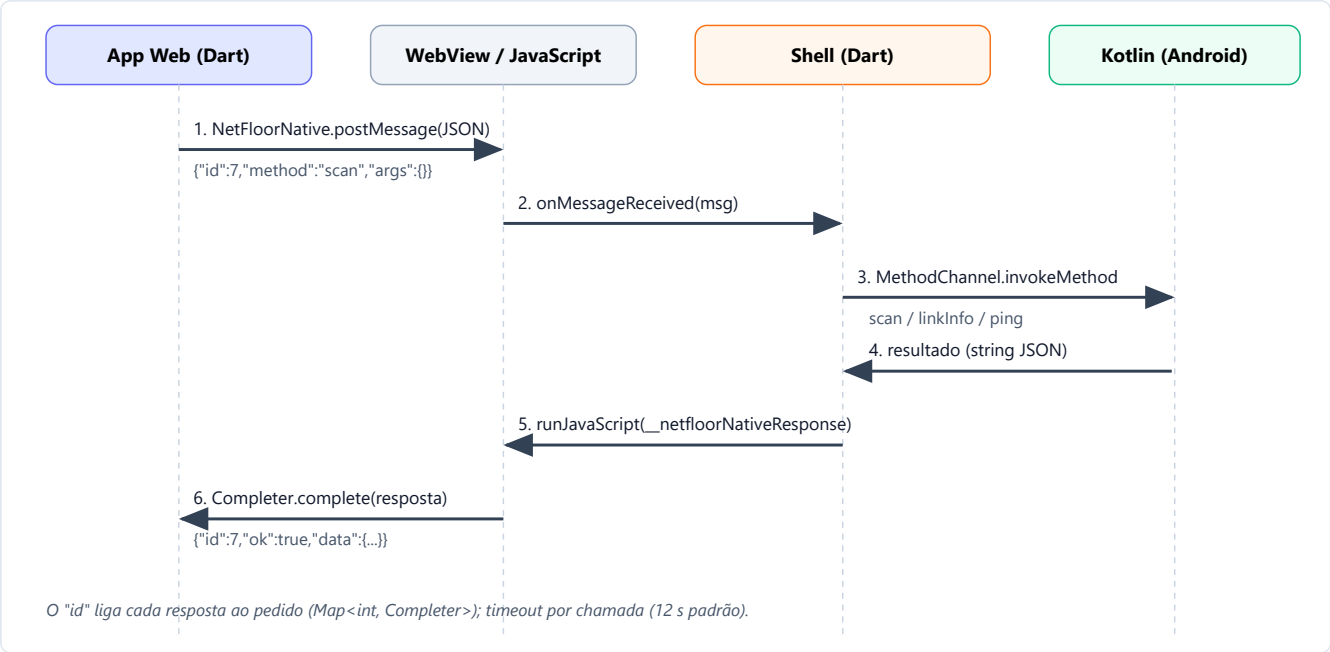
2.4 Android Shell

O Shell é um aplicativo Flutter mínimo cujo corpo é uma `WebView` em tela cheia.

Responsabilidade	Implementação
Carregar o app	`WebViewController.loadRequest` para `https://rogerdev5690.github.io/netfloor/`
Proteger a ponte	`onNavigationRequest` só permite o host `rogerdev5690.github.io`; qualquer outra navegação é bloqueada
Tratar falha de rede	Tela "Não foi possível carregar" com botão de nova tentativa (apenas para erros do quadro principal)
Expor a ponte	`addJavaScriptChannel('NetFloorNative', ...)`
Permissões em tempo de execução	`permission_handler`: `locationWhenInUse` e `nearbyWifiDevices`
Seletor de arquivos	`AndroidWebViewController.setOnShowFileSelector` chama `FilePicker.pickFile` / `pickFiles` e devolve URIs; quando a página pede `.json` (importação de projeto), abre o seletor filtrado por JSON
Entrega de arquivos (3.2.0)	Método `saveFile` da ponte: remonta o arquivo recebido em pedaços de base64 e o entrega ao Kotlin, que grava em **Downloads/WaveLens** (`MediaStore`) ou abre a **folha de compartilhamento** (`FileProvider`)
Acesso ao Wi-Fi	`MethodChannel('netfloor/diag')` para o Kotlin

2.5 Ponte de Comunicação (JS Bridge)

A ponte é **bidirecional** e assíncrona, baseada em mensagens JSON com identificador de correlação.



Sequência da ponte Kotlin ⇄ Flutter Web

Formato das mensagens

```
// App Web -> Shell (NetFloorNative.postMessage)
{ "id": 7, "method": "ping", "args": { "host": "8.8.8.8" } }

// Shell -> App Web (window.__netfloorNativeResponse)
{ "id": 7, "ok": true,  "data": { "ms": 21.4 } }
{ "id": 7, "ok": false, "error": "mensagem de erro" }
```

Métodos disponíveis

Método	Argumentos	Resposta (`data`)	Timeout no app web
`hello`	—	`{ platform, shellVersion }`	12 s
`permissions`	—	`{ location, nearby, locationServices }` (booleanos)	90 s (usuário decide)
`scan`	—	`{ networks: [...], scanStarted, error? }`	20 s
`linkInfo`	—	`{ connected, ssid, bssid, rssi, linkSpeed, frequency, gateway, ip }`	12 s
`ping`	`host`	`{ ms }` (`null` se perdido)	8 s
`saveFile`	`name`, `mime`, `share`, `chunk`, `chunks`, `data` (base64)	`{ done }` e, no último pedaço, `{ done: true, location }`	60 s por pedaço

Como funciona.

1. No lado web, `NativeBridge.available` é verdadeiro quando existe `window.NetFloorNative` (injetado pela WebView do Shell). Cada chamada gera um `id` incremental e guarda um `Completer` em um mapa.
2. A resposta chega pela função global `window.__netfloorNativeResponse`, registrada pelo próprio app web na primeira chamada. O `id` localiza o `Completer` correspondente.
3. No Shell, a resposta é serializada duas vezes (`jsonEncode(jsonEncode(...))`) para formar um literal de string JavaScript válido antes de `runJavaScript`.
4. Estourando o tempo limite, a chamada lança `TimeoutException` e a UI mostra o erro da respectiva aba.

Envio de arquivos (`saveFile`). Mensagens grandes na ponte JS são frágeis; por isso o app web divide o arquivo em pedaços de 256 KB de base64 (`chunk` 0 a `chunks - 1`). O Shell acumula os pedaços e, no último, decodifica os bytes e os passa por `MethodChannel` ao Kotlin. Com `share = false` o arquivo vai para `Downloads/WaveLens` (Android 10+: `MediaStore.Downloads`; antes: pasta de downloads do app, sem exigir permissão de armazenamento); com `share = true` é copiado para o cache e aberto na folha de compartilhamento (WhatsApp, e-mail, Drive…). O app web só usa esse caminho se `hello` informar Shell ≥ 3.2.0; em navegador comum faz o download normal por `Blob`.

Segurança. A ponte só existe na WebView do Shell, e o Shell bloqueia navegação para domínios diferentes do GitHub Pages do projeto. Métodos desconhecidos são rejeitados (`UnsupportedError`). O `ping` valida o nome do host por expressão regular antes de executar.

2.6 Ícone do aplicativo

O ícone (launcher do Android e favicon/PWA do site) é um arquivo único, `assets/icon/icon.png` — **1024×1024, PNG, sem cantos arredondados** (o próprio Android/navegador aplica a máscara/arredondamento na hora de exibir).

Onde	Gerado a partir de	Como regenerar
`netfloor/web/icons/Icon-192.png`, `Icon-512.png`, `Icon-maskable-*.png`, `web/favicon.png`	`netfloor/assets/icon/icon.png` (redimensionado)	Script Python com Pillow (`Image.resize`), rodado manualmente
`netfloor_shell/android/app/src/main/res/mipmap-*/ic_launcher.png` (5 densidades)	`netfloor_shell/assets/icon/icon.png`	Pacote `flutter_launcher_icons` (dev dependency), configurado em `netfloor_shell/pubspec.yaml`: `flutter pub run flutter_launcher_icons` (ou `dart run flutter_launcher_icons`)

Para trocar o ícone: substitua os dois arquivos `assets/icon/icon.png` (idênticos, um em cada repositório) por uma imagem quadrada de alta resolução (1024×1024 recomendado, fundo sólido até a borda, sem transparência nas bordas) e rode novamente o gerador do Shell + o script de redimensionamento do web, depois publique (seção 7.6) e gere um novo APK (seção 7.7, obrigatório sempre que o ícone/launcher muda — é um recurso nativo, não chega por OTA).

3. Motor do Simulador e Engenharia de RF

3.1 Fórmula de propagação

Para cada ponto (x, y) do pavimento exibido, o sinal é o **máximo** entre os roteadores da rede (união da cobertura de uma rede mesh):

$$S(x,y) = \max_i \left(P_{tx,i} + \Delta_{ref}(b) - k(b) \cdot \log_{10}(d_{3D,i}) - f_{par}(b) \cdot \sum L_{parede} - L_{laje,i} \right)$$

Onde:

Símbolo	Significado
$P_{tx,i}$	Potência de transmissão do roteador i, em dBm, definida pelo modelo (seção 3.2)
b	Banda simulada: 2.4, 5 ou 6 GHz (seção 3.10)
$\Delta_{ref}(b)$	Ajuste do nível a 1 m em relação a 2.4 GHz (0, −6,4 e −7,9 dB)
$k(b)$	Coeficiente de perda de percurso: $10 \cdot n$ (22, 24 e 25)
$d_{3D,i}$	Distância tridimensional entre o roteador e o ponto, em metros reais (mínimo de 1 m)
$f_{par}(b)$	Fator de opacidade das paredes na banda (1,00, 1,35 e 1,50)
$\sum L_{parede}$	Soma da atenuação das paredes cruzadas pelo raio entre roteador e ponto (seção 3.4)
$L_{laje,i}$	Perda por laje: 15 dB para cada pavimento de diferença (não depende da banda)

Em 2.4 GHz a expressão se reduz à fórmula original do projeto, $P_{tx} - 22\log_{10}(d) - \sum L_{parede} - L_{laje}$ (coeficiente 22 = expoente 2,2).

3.2 Catálogo de hardware

Modelo	Potência base (P_{tx})	Ícone (desenhado por `CustomPainter`)
Huawei AX3 Pro / AX3s	18 dBm	Roteador de mesa, corpo escuro, 4 antenas externas, LED verde
TP-Link Deco (Mesh)	23 dBm	Torre/cilindro branco minimalista, sem antenas
Ubiquiti UniFi (AP Pro)	28 dBm	Disco de teto circular com anel de luz azul central
ZTE E2320 / E2620 (ZXHN, Wi-Fi 6)	20 dBm	ONT/roteador branco de mesa, duas antenas externas inclinadas, faixa de LEDs

A escolha do modelo é feita por uma `BottomSheet` ao tocar em **Adicionar Roteador**; o modelo selecionado também é o padrão para roteadores inseridos por toque na planta.

O ZTE E2320 foi cadastrado com potência (20 dBm) **acima do Huawei AX3** (18 dBm), refletindo o maior ganho/EIRP típico de um CPE Wi-Fi 6 com beamforming — na simulação, cobertura e alcance maiores que o Huawei nas mesmas condições de parede e distância.

3.3 Distância 3D e escala

Até a v7.0 a escala era fixa (45 px por metro). Na v8.0 cada pavimento tem a **largura real** W_m da planta, em metros (`FloorState.widthM`), e todo o cálculo é feito em **metros**, independentemente do tamanho da tela. As posições dos roteadores, pontos e paredes continuam guardadas como **frações** (0 a 1) da planta:

$$x_m = x \cdot W_m, \qquad y_m = y \cdot \frac{W_m}{r}, \qquad r = \frac{\text{largura}}{\text{altura}} \text{ (proporção da imagem)}$$

$$d_{3D} = \max\left(\sqrt{(\Delta x_m)^2 + (\Delta y_m)^2 + (\Delta a \cdot h)^2},\ 1\right)$$

Onde Δa é a diferença de andares (0 = mesmo andar) e $h = 3$ m (`kFloorHeightM`).

Calibração pela régua. O usuário traça uma linha sobre uma medida conhecida (por exemplo, uma parede) e informa a distância real D. Com ℓ o comprimento da linha em "larguras da planta",

$$\ell = \sqrt{(\Delta x)^2 + \left(\frac{\Delta y}{r}\right)^2}, \qquad W_m = \frac{D}{\ell}$$

O novo W_m substitui o valor estimado e o mapa de calor é recalculado. Enquanto o pavimento não é calibrado, vale `defaultWidthM` da planta (8 m para a Casa 2 Quartos, 12 m para as plantas 01 e 02, 10 m para a 03 e para uploads, 20 m para o edifício) e o selo de escala mostra "(estimada)". Cada pavimento tem sua própria calibração.

3.4 Física de barreiras (ray-casting)

Cada planta tem uma lista de **segmentos de parede** (`WallSegment`) em coordenadas fracionárias, com uma atenuação em dB. Para cada célula da grade e cada roteador, o motor traça o segmento de reta entre eles e soma a atenuação de **todas as paredes que o raio cruza**.

O teste de interseção usa a orientação de pontos (caso geral):

```
bool _segmentsIntersect(Offset p1, Offset p2, Offset p3, Offset p4) {
  double orient(Offset a, Offset b, Offset c) =>
      (b.dx - a.dx) * (c.dy - a.dy) - (b.dy - a.dy) * (c.dx - a.dx);
  final o1 = orient(p1, p2, p3), o2 = orient(p1, p2, p4);
  final o3 = orient(p3, p4, p1), o4 = orient(p3, p4, p2);
  return ((o1 > 0) != (o2 > 0)) && ((o3 > 0) != (o4 > 0));
}
```

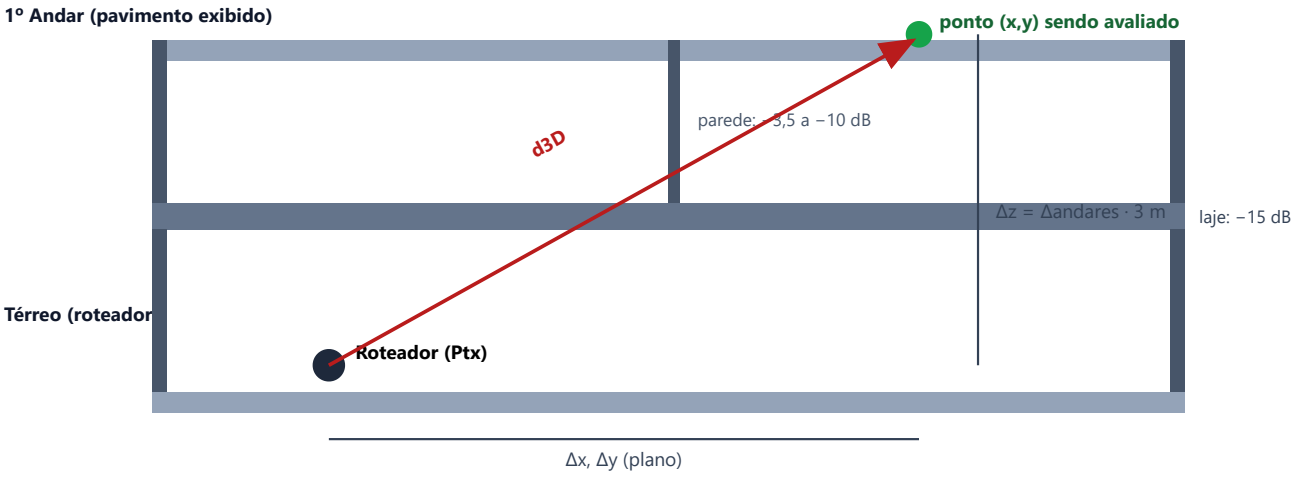
Tipo de barreira	Atenuação usada (2.4 GHz)	Onde aparece
Parede leve / divisória de varanda	3,5 dB	Varandas e terraços (plantas da biblioteca)
Parede interna padrão	6,0 dB (padrão)	Divisórias entre cômodos (plantas da biblioteca)
Parede pesada	10,0 dB	Parede central do escritório
Gesso / Drywall	2 dB	Paredes desenhadas pelo usuário
Tijolo cerâmico	4 dB	Paredes desenhadas pelo usuário
Vidro / Espelho	5 dB	Paredes desenhadas pelo usuário
Concreto armado / Laje	12 a 15 dB (padrão 13,5; ajustável)	Paredes desenhadas pelo usuário
Laje de concreto entre andares	15 dB por andar	Entre pavimentos

As paredes de cada planta da biblioteca são **aproximações** desenhadas a partir da imagem (não medidas a laser). O usuário pode **desenhar paredes próprias** com a ferramenta *Parede* (seção 5.3), escolhendo o material; elas valem para qualquer planta, inclusive as enviadas do dispositivo, e somam-se às da biblioteca. As paredes usadas no cálculo de um andar são as do pavimento exibido. Nas bandas de 5 e 6 GHz a atenuação de cada parede é multiplicada por $f_{par}(b)$.

3.5 Arquitetura 2.5D multi-pavimento

Um **projeto** (`ProjectDef`) tem um ou mais **pavimentos** (`FloorDef`), e cada pavimento usa uma planta (`FloorPlanDef`).

- **Seletor de pavimento:** uma linha de chips (Térreo, 1º Andar, 2º Andar…) acima da planta, com o número de roteadores de cada andar.
- **Botão "+ Pavimento":** acrescenta andares ao projeto atual a partir de imagens do dispositivo.
- **Roteadores de outros andares** aparecem esmaecidos (opacidade 0,55, sem interação) no andar exibido, indicando a origem do sinal.
- **Suposição:** os pavimentos são empilhados sobre a mesma pegada, isto é, a mesma posição fracionária (x, y) em cada andar.



Propagação 2.5D: distância 3D, paredes e laje

3.6 Exemplos numéricos

Caso	Cálculo	Sinal	Cor aproximada
Huawei (18 dBm), 4 m, mesmo andar, 1 parede de 6 dB	$18 - 22\log_{10}(4) - 6 = 18 - 13{,}25 - 6$	−1,2 dBm	verde
Mesmo Huawei, ponto logo acima (1 andar, d_{3D} = 3 m)	$18 - 22\log_{10}(3) - 15 = 18 - 10{,}50 - 15$	−7,5 dBm	ciano/azulado
UniFi (28 dBm), 10 m, mesmo andar, sem paredes	$28 - 22\log_{10}(10) = 28 - 22$	+6,0 dBm	verde/amarelo

3.7 Visualização do mapa de calor

Amostragem. A planta é dividida em uma grade de células de 8 px (no laudo, o mesmo motor renderiza uma grade proporcional em imagem de 1000 px de largura). Para cada célula avalia-se $S(x, y)$ por meio de `SignalField.at` e converte-se em cor e transparência.

Escala. O sinal é normalizado por $t = \text{clamp}\big((S + 30) / 56,\ 0,\ 1\big)$, ou seja, −30 dBm mapeia para $t = 0$ e +26 dBm para $t = 1$ (próximo à potência máxima do catálogo).

t	Cor (paleta "jet")
0,00	Azul profundo `#1E3A8A`
0,18	Azul `#2563EB`
0,38	Ciano `#06B6D4`
0,58	Verde `#22C55E`
0,74	Amarelo `#FACC15`
0,88	Laranja `#FB923C`
1,00	Vermelho `#EF4444`

Transparência. A opacidade cresce suavemente (interpolação *smoothstep* entre $t = 0{,}04$ e $t = 0{,}55$) até o máximo configurável. O padrão é **48 %**; o usuário ajusta entre 15 % e 90 % em uma folha inferior (ícone de gota na barra superior). No Modo Diagnóstico (seção 5.4) a opacidade exibida é multiplicada por 1,25 (limite de 95 %). Regiões sem cobertura ficam transparentes, deixando a planta visível.

Suavização. A camada é desfocada (`ImageFilter.blur`, sigma 9) para remover o aspecto quadriculado.

Legenda. Os limiares são as constantes `kStrongDbm = 12` e `kWeakDbm = -10`: **Forte** ≥ 12 dBm; **Intermediário** de −10 a 12 dBm; **Ruim** < −10 dBm. A legenda fica sobre o mapa (fixa, sem acompanhar o zoom); no laudo aparece também uma faixa contínua de −30 a +26 dBm.

3.8 Interação

O mapa fica dentro de um `InteractiveViewer` (zoom de 1× a 12×). Como o `InteractiveViewer` e os gestos de arrastar disputam o mesmo toque, toda a interação com os elementos do mapa usa **ponteiros crus** (`Listener`), que não entram na disputa de gestos; um segundo dedo cancela o desenho em curso e deixa o `InteractiveViewer` tratar o pinch. Detalhes na seção 5.3.

Ação	Comportamento
Pinch com 2 dedos / roda do mouse / botões + e −	Zoom e deslocamento do mapa; o botão "ajustar à tela" volta a 1×
Ferramenta *Roteador*: tocar em área livre	Insere um roteador do modelo atual naquele ponto
Botão "Roteador" (barra inferior)	Abre a seleção de modelo e insere o roteador no centro (com pequeno deslocamento para não sobrepor)
Ferramenta *Roteador*: arrastar um roteador	Move o roteador dentro dos limites da planta; o mapa de calor é recalculado a cada quadro. Arrastar em área livre desloca o mapa (quando ampliado)
Tocar em um roteador existente	Não cria um roteador duplicado
Botão de lixeira	Remove todos os roteadores (de todos os andares)
Anéis pulsantes	Dois anéis expansivos por roteador (ciclo de 2,4 s) sugerem a emissão de sinal; roteadores e anéis mantêm tamanho constante na tela, mesmo com zoom

3.9 Limitações do modelo

- É um modelo **simplificado** de perda de percurso; não modela multipercurso, difração, interferência entre roteadores nem a orientação das antenas. A frequência é considerada de forma **aproximada** (três bandas, seção 3.10) e os materiais são valores típicos, não medições.
- O nível "dBm" do simulador é uma escala **relativa** (potência de transmissão menos perdas, com o nível de referência a 1 m igual à potência do modelo); **não é** o RSSI que um aparelho mediria. Por isso o laudo mostra os dois lado a lado, sem compará-los numericamente (seção 5.7).
- A escala só é confiável depois de calibrada com a régua; sem isso vale a largura estimada da planta.
- As paredes das plantas da biblioteca são aproximações visuais; plantas enviadas pelo usuário só têm as paredes que ele desenhar.
- Em imagens em perspectiva 3D as paredes do cálculo são apenas indicativas.
- Os pavimentos são empilhados sobre a mesma pegada (mesma posição fracionária em cada andar), e a perda de laje é fixa (15 dB) em todas as bandas.

3.10 Simulação em 2.4 / 5 / 6 GHz

Um alternador de banda (chips **2.4 GHz · 5 GHz · 6 GHz**) refaz o mapa de calor na hora. A referência é 2.4 GHz; nas bandas mais altas o alcance diminui por três efeitos, todos parametrizados em `kRfBands`:

Banda	Δ_{ref} (dB, a 1 m)	k (= 10·n)	f_{par} (paredes)
2.4 GHz	0	22	1,00
5 GHz	−6,4	24	1,35
6 GHz	−7,9	25	1,50

O ajuste Δ_{ref} é a diferença de perda em espaço livre a 1 m, $20\log_{10}\left(f\,/\,2{,}4\,\mathrm{GHz}\right)$, para 5,0 e 5,95 GHz. O expoente maior e a opacidade maior das paredes representam a pior propagação em obstáculos. Os valores são estimativas de engenharia para

comparação relativa entre bandas, não medições. A banda escolhida é salva com o projeto e usada no laudo (que pode incluir várias bandas).

4. Módulo de Diagnóstico Mobile (Android Nativo)

4.1 Modos de operação

Modo	Condição	Dados	Indicação na tela
Nativo	`window.NetFloorNative` existe (WebView do Shell)	Reais, do aparelho	Faixa verde: "Dados reais do aparelho Android."
Simulação	Navegador comum ou PWA	Fictícios	Faixa amarela: "Modo simulação: dados fictícios..."

A fonte de dados é escolhida por `DiagnosticsController`: começa em `SimulatedDiagnostics` e troca para `NativeBridgeDiagnostics` assim que a ponte é detectada, limpando os dados anteriores. Ambas implementam `DiagnosticsSource` (`ensurePermissions`, `scan`, `linkInfo`, `ping`).

4.2 Requisitos e permissões do Android

Requisito	Motivo
Permissão de **Localização** concedida	O Android só devolve redes Wi-Fi e SSID/BSSID a apps com localização
GPS / Localização do aparelho ligada	Sem ela `getScanResults` retorna vazio a partir do Android 9
`NEARBY_WIFI_DEVICES` (Android 13+)	Alternativa moderna de permissão para varredura
Limite de varreduras	O Android limita `startScan` (cerca de 4 a cada 2 minutos em primeiro plano); nesse caso retorna o último resultado em cache

Se a permissão for negada, ou a localização estiver desligada, um cartão laranja explica o problema e oferece o botão **Conceder**.

4.3 Aba Espectro e Saúde do Canal

Coleta. O Kotlin chama `WifiManager.startScan()` e lê `scanResults`. Para cada rede são enviados: SSID, BSSID, frequência do canal primário, frequência central do bloco (`centerFreq0`), largura (20/40/80/160 MHz), nível em dBm e um indicador `connected` (comparação do BSSID com o da conexão atual). Redes fora de 2.4 GHz e 5 GHz (por exemplo 6 GHz) são descartadas da aba Espectro. Já a medição de campo (seção 5.5) rotula corretamente uma conexão em 6 GHz (frequência ≥ 5925 MHz).

Relação canal ⇄ frequência.

$$f_{2,4}(c) = 2407 + 5c \quad (c = 1..13;\, c = 14 \Rightarrow 2484\ \text{MHz}), \qquad f_5(c) = 5000 + 5c$$

Desenho. `SpectrumPainter` desenha um **trapézio por rede**: a base tem a largura do canal, a altura é proporcional ao nível (eixo de −100 a −30 dBm) e o topo é 10 % mais estreito. O eixo X cobre 2396–2498 MHz (2.4 GHz) e 5160–5840 MHz (5 GHz). No 5 GHz o gráfico tem largura mínima de 980 px e rola horizontalmente para que todos os canais fiquem legíveis.

Elemento	Estilo
Rede conectada	Verde aceso (`#00E676`) com brilho, preenchimento, contorno, **barra sublinhada no eixo** e rótulo sublinhado com SSID e dBm
Mesmo SSID em outra banda	Tratado como "sua rede": ex.: você está no 5 GHz e o 2.4 GHz do mesmo roteador aparece em destaque com o sufixo "· sua rede"
Redes vizinhas	Cinza translúcido; rótulos só das 4 mais fortes, para evitar poluição

Um cartão **Rede conectada** mostra SSID, BSSID, banda e canal, largura em MHz e nível em dBm. Se o Android ocultar o BSSID (sem permissão), o campo exibe "oculto pelo Android".

Saúde do canal. Para cada canal de 20 MHz calcula-se uma pontuação de interferência a partir das redes vizinhas. A sua própria rede (conectada e as de mesmo SSID) fica **de fora**.

$$o_{c,n} = \text{clamp}\left(\frac{\min(f_c + 10,\ hi_n) - \max(f_c - 10,\ lo_n)}{20},\ 0,\ 1\right), \qquad s_n = \text{clamp}\left(\frac{RSSI_n + 95}{55},\ 0,\ 1\right)$$

$$\text{score}_c = \sum_n o_{c,n} \cdot s_n$$

Onde $[lo_n, hi_n]$ é o bloco de frequências ocupado pela rede n (centro ± largura/2).

Pontuação	Classificação
< 0,25	**Excelente**
0,25 a < 1,0	**Bom**
≥ 1,0	**Ruim**

São avaliados os canais 1–13 (2.4 GHz) e 25 canais de 5 GHz (36–64, 100–144, 149–165). A tela lista os **3 canais recomendados** (menor pontuação) e uma tabela completa com canal, frequência, barra de interferência, número de redes sobrepostas e classificação.

Atualização. A varredura roda ao abrir a aba e a cada 30 s enquanto ela estiver visível; há também o botão **Escanear**.

4.4 Aba Sinal (walk-through)

O aplicativo lê o RSSI da conexão atual a cada **1 segundo** e mantém as últimas 60 amostras em um gráfico de linha (`fl_chart`). Linhas tracejadas de referência marcam −60 dBm e −70 dBm. A tela mostra o valor atual, SSID, banda e canal, além de mínimo, média e máximo da janela.

RSSI	Classificação
≥ −50 dBm	Excelente
−50 a −60 dBm	Muito bom
−60 a −70 dBm	Bom
−70 a −80 dBm	Fraco
< −80 dBm	Muito fraco

Os botões **Pausar/Retomar** e **Limpar** controlam a coleta. O uso típico é caminhar pelo ambiente e observar as quedas do gráfico.

4.5 Aba Latência e Rede (ping duplo)

A cada segundo o aplicativo mede, **em paralelo**, a latência até:

1. **Gateway local**: IP do roteador obtido de `DhcpInfo.gateway` (atualizado a cada 5 ciclos).
2. **DNS da Internet**: `8.8.8.8`.

Indicador	Origem
Perda de pacotes (%)	$\text{perdidos} / \text{enviados} \times 100$, por alvo
Latência DNS (ms)	Última medição e média
Latência do gateway (ms)	Última medição, média e IP
Velocidade PHY (Mbps)	`WifiInfo.getLinkSpeed()` (taxa negociada com o roteador)

Implementação do ping no Android. O Kotlin executa `/system/bin/ping -c 1 -W 2 <host>` e extrai o valor `time=` da saída. Se o binário não puder ser executado, usa-se um **fallback TCP**: mede-se o tempo de `connect` nas portas 53 e 80 (conexão recusada também prova alcance). O nome do host é validado antes da execução.

Interpretação. Se só o DNS piora, o problema tende a estar fora de casa; se o gateway também piora, o problema é na rede local (Wi-Fi). Pacotes perdidos aparecem como lacunas no gráfico.

4.6 Controlador de diagnóstico

`DiagnosticsController` executa apenas o trabalho da **aba visível**, poupando bateria e respeitando os limites do Android.

Aba	Temporizador	Proteções
Espectro	Varredura imediata + a cada 30 s	Não inicia nova varredura enquanto outra roda
Sinal	A cada 1 s (se "Retomar")	Ignora ciclos enquanto a leitura anterior não terminou
Latência	A cada 1 s (se "Retomar")	Idem; os dois pings rodam com `Future.wait`

Ao trocar de aba ou de tela, os temporizadores são cancelados e recriados conforme necessário. As amostras permanecem guardadas quando o usuário alterna entre abas.

4.7 Modo simulação

`SimulatedDiagnostics` gera 13 redes fictícias (2.4 e 5 GHz, larguras variadas, uma rede conectada e o 2.4 GHz de mesmo nome), um RSSI que oscila como uma caminhada, IP de gateway `192.168.0.1`, latências plausíveis e ~3 % de perda. Serve para demonstrar a interface em qualquer navegador.

4.8 Estado de validação

Item	Situação (na data desta revisão)
Modo simulação (3 abas)	Verificado no navegador
Ponte com dados no formato nativo	Verificada injetando uma ponte simulada no navegador
Dados reais no Android	**Validado em aparelho real** (Xiaomi Redmi Note 14 Pro): espectro 2.4/5 GHz, gateway `192.168.100.1`, PHY 390 Mbps
Seleção múltipla de imagens no Shell 3.1.0	Compilado; ainda não validado em aparelho
Testes automatizados	Não há (o app usa `dart:js_interop`, que não executa na VM de testes)

5. WaveLens Enterprise (v8.0)

5.1 Visão geral

Recurso	Resumo	Seção
Ferramentas do mapa	Roteador, **Parede** (com materiais), **Régua** (calibração de escala) e **Ponto** (medição)	5.3
Zoom e deslocamento	`InteractiveViewer` com pinch, roda do mouse e botões	5.3
Modos de visualização	**Apresentação** (escuro/moderno) e **Diagnóstico** (alto contraste)	5.4
Medição de campo	RSSI, PHY, ping ao gateway e à Internet, perda de pacotes	5.5
Projetos offline	Salvamento automático em IndexedDB; exportar/importar `.json`	5.6
Laudo de vistoria	PDF com logo, mapas de calor por banda, tabelas e assinatura do cliente	5.7 e 5.8
Entrega de arquivos	Download no navegador; salvar/compartilhar no Shell 3.2+	5.9

A banda de operação (2.4/5/6 GHz) e a física de paredes/escala estão nas seções 3.3, 3.4 e 3.10.

5.2 Fluxo de trabalho típico em campo

1. Abrir uma planta da biblioteca ou **carregar a planta do cliente** (foto/imagem).
2. Com a **Régua**, medir uma parede conhecida e informar a distância real (calibra a escala).
3. Com **Parede**, desenhar as paredes reais escolhendo o material.
4. Posicionar os roteadores e comparar as bandas 2.4/5/6 GHz.
5. Marcar **pontos** nos cômodos críticos e, no local, tocar em **Medir agora** (ou usar *Registrar medição* na aba Diagnóstico).
6. Abrir o **Laudo**, preencher os dados, coletar a **assinatura** do cliente e gerar o PDF.

Tudo é salvo automaticamente no aparelho; o projeto pode ser retomado depois e exportado em `.json`. **Atenção:** os *dados* ficam offline, mas o *aplicativo* ainda é carregado do GitHub Pages (`--pwa-strategy=none`, base do OTA), então é preciso ter internet ao abrir o app (ver seção 8).

5.3 Ferramentas do mapa e gestos

Ferramenta	Gesto	Resultado
Roteador	Toque em área livre; arrastar um roteador	Insere / move roteadores (seção 3.8)
Parede	Arrastar no mapa; tocar em uma parede	Traça uma parede (com **encaixe** em horizontal/vertical a menos de ~4° e nos cantos de outras paredes); tocar abre a folha para trocar o material, ajustar a perda do concreto (12 a 15 dB) ou remover
Régua	Arrastar sobre uma medida conhecida	Mostra "≈ X m" durante o arraste e abre o diálogo de distância real; confirma e recalcula a escala
Ponto	Tocar no mapa; tocar em um ponto; arrastar	Cria/edita/move um ponto de medição (nome, previsão, "Medir agora", remover)

Materiais de parede (cores no mapa): Gesso/Drywall −2 dB (verde), Tijolo cerâmico −4 dB (âmbar), Vidro/Espelho −5 dB (ciano), Concreto armado/Laje −12 a −15 dB (vermelho). As paredes aproximadas da biblioteca aparecem tracejadas em cinza quando a ferramenta *Parede* está ativa.

Zoom e gestos. Dois dedos fazem pinch/pan (o `InteractiveViewer` cuida disso). O deslocamento com um dedo só fica ativo nas ferramentas *Roteador* e *Ponto*, e apenas quando o toque não começa sobre um marcador. Nas ferramentas de desenho, um dedo desenha e dois dedos navegam. Os traços, marcadores e anéis compensam o zoom (`1/zoom`) para manter tamanho constante na tela.

Por que `Listener`? O reconhecedor de escala do `InteractiveViewer` aceita o gesto (a partir de ~18 px) antes do `onPanUpdate` de um filho (~36 px), o que impediria arrastar roteadores. Ponteiros crus (`onPointerDown/Move/Up`) recebem todos os eventos sem disputar a arena de gestos; a lógica de toque (limite de 8 px de movimento e 600 ms) e de arraste é própria. Cancelar por segundo dedo evita que um pinch deixe uma parede "pela metade".

5.4 Modos de visualização

Modo	Aparência	Uso
Apresentação (padrão)	Tema escuro (`#0B1020`), cartões arredondados, planta com filtro que inverte a luminosidade e preserva os matizes	Mostrar a simulação ao cliente
Diagnóstico	Alto contraste: fundo branco, barra preta, chips amarelos, contornos pretos, texto ≥ 1,08×; planta em tons de cinza com contraste e mapa de calor 25 % mais opaco	Uso externo sob sol forte

O botão de contraste na barra do simulador alterna os modos; a escolha é salva (`settings.viewMode`). O filtro só afeta a **exibição**: o laudo em PDF sempre usa a planta original, própria para impressão.

5.5 Pontos de medição e medição de campo

`DiagnosticsController.captureSnapshot` (5 pares de ping por padrão) devolve um `Measurement`:

Campo	Origem
`rssi`, `linkMbps` (PHY), `ssid`, `bssid`, banda/canal	`linkInfo` (WifiInfo)
`gwAvgMs`, `gwLossPct`	Ping ao gateway (DHCP)
`netAvgMs`, `netLossPct`	Ping ao DNS público `8.8.8.8`
`native`	`true` só quando a fonte é o Shell Android; `false` = **simulado**

Uma medição pode ser vinculada a um **ponto** (o ponto guarda a última) e sempre entra no **histórico do projeto** (`logs`), que é salvo e exportado. No mapa, um ponto medido tem uma bolinha branca no centro; a cor do ponto é a classe prevista (verde/âmbar/vermelho). Fora do Shell os dados são simulados e assim marcados no histórico e no laudo (aviso em vermelho), para nunca serem confundidos com a rede real do cliente.

5.6 Persistência offline-first e arquivo `.json`

Armazenamento local. `AppStore` usa `sembast_web` (IndexedDB) com o banco `netfloor_v8`:

Store	Chave	Conteúdo
`workspaces`	id do projeto	Documento JSON do projeto (sem bytes de imagem)
`images`	id da planta personalizada	Imagem em base64
`settings`	nome	`viewMode`, `lastWorkspace`

O salvamento é automático, 0,8 s após a última alteração, e imediato quando a página é escondida (`visibilitychange`). Ao abrir o app, o último projeto é restaurado. A folha **Projetos e plantas** lista os projetos salvos (com exclusão confirmada), as plantas da biblioteca, o upload de plantas e a importação. Se o IndexedDB estiver indisponível (por exemplo, aba anônima), o app segue funcionando sem persistir e avisa na folha de projetos. A WebView do Shell mantém o DOM Storage ligado por padrão, então o armazenamento também funciona no aplicativo Android.

Exportar/importar. *Exportar projeto (.json)* gera `<nome>.netfloor.json` (indentado, UTF-8) com **tudo**: plantas personalizadas embutidas em base64, escala e calibração, paredes, roteadores, pontos, medições, dados do laudo (inclusive logo e assinatura) e banda. Plantas da biblioteca entram só como referência (`planId`), o que mantém o arquivo leve (um projeto típico tem ~25 KB). *Importar* cria um **novo** projeto local (novo id, novos ids de imagem), sem sobrescrever o existente.

```
{
  "format": "netfloor-project", "schema": 1, "appVersion": "8.0",
  "id": "...", "name": "...", "updatedAt": "...", "projectId": "casa_2q",
  "band": "ghz5", "model": "huaweiAx3", "heatOpacity": 0.48,
  "floors": [ { "label": "Térreo", "planId": "casa_2q", "custom": false,
                "widthM": 13.3, "calibrated": true,
                "walls": [ { "ax": 0.27, "ay": 0.42, "bx": 0.74, "by": 0.42, "db": 2, "mat": "drywall" } ] } ],
  "routers": [ { "id": "r0", "x": 0.67, "y": 0.63, "floor": 0, "model": "huaweiAx3" } ],
  "points":  [ { "id": "p...", "name": "P1", "x": 0.24, "y": 0.67, "floor": 0, "measured": { } } ],
  "logs": [ ], "report": { }
}
```

`applyJson` valida o formato (`format = netfloor-project`), a existência das plantas e das imagens e rejeita arquivos inválidos com mensagem clara.

5.7 Laudo de vistoria em PDF

A tela **Laudo de vistoria** (ícone de documento na barra do simulador) reúne os dados do laudo, que são salvos com o projeto:

Campo	Uso no PDF
Técnico responsável, Empresa/ISP	Cabeçalho, identificação e bloco de assinatura do técnico
Logo da empresa (PNG/JPG/WEBP enviado)	Canto superior esquerdo da capa
Cliente, Endereço, Data	Identificação (a data padrão é o dia atual)
Bandas com mapa de calor	Uma página por pavimento **e** banda escolhida
Observações do técnico	Caixa de texto livre
Assinatura do cliente	Bloco final (seção 5.8)

Conteúdo do PDF (A4).

1. **Capa e resumo:** identificação, resumo da simulação (pavimentos, roteadores, bandas, se a escala foi calibrada, pontos críticos, medições) e tabela de roteadores (modelo, andar, potência, posição em metros).
2. **Mapas de calor** (uma página por pavimento e banda): planta + calor + paredes desenhadas + roteadores numerados (R1, R2…) + pontos com o valor previsto + barra de escala; **legenda detalhada de dBm** (faixa contínua de −30 a +26 dBm e as classes Forte/Intermediário/Ruim) e o resumo de cobertura do andar (% forte, intermediária e ruim, pior ponto com coordenadas em metros).
3. **Diagnóstico e pontos críticos:** cobertura por pavimento e banda; tabela **previsto × medido** (ponto, andar, nível simulado, classe, RSSI, PHY, ping ao gateway, ping à Internet, perda, status); registro de diagnósticos de campo; observações do técnico; assinaturas.

Status dos pontos. *CRÍTICO*: classe Ruim (< −10 dBm simulados), RSSI ≤ −75 dBm, perda ≥ 5 % ou ping ao gateway > 30 ms. *Atenção*: classe Intermediária, RSSI ≤ −67 dBm, alguma perda ou ping ao gateway > 15 ms. Caso contrário, *OK*. O PDF traz uma nota explicando que o **nível simulado (escala relativa) e o RSSI medido não são numericamente comparáveis**, e um aviso em destaque quando existem medições simuladas.

Como é gerado.

Etapa	Implementação
Renderização do mapa	`renderFloorRaster`: `PictureRecorder` + o mesmo `HeatmapPainter` / `SignalField` do simulador, com desfoque por `saveLayer` (`ImageFilter.blur`), roteadores por `RouterDevicePainter`, pontos por `PointMarkerPainter`; largura de 1000 px
Compressão	A imagem é codificada em **JPEG** (qualidade 88, `package:image`) e incorporada ao PDF **sem recompressão**; um laudo típico com duas bandas fica em torno de 1 MB
Documento	Biblioteca `pdf` (100 % Dart, roda no navegador e na WebView): `MultiPage` / `Page` A4, tabelas com `TableHelper`, rodapé com paginação
Fonte	Roboto Regular/Bold/Italic embutida em `assets/fonts/` (acentos, "≥", "−")

O laudo é gerado localmente; nada é enviado a servidores.

5.8 Assinatura digital do cliente

O botão **Coletar assinatura** abre uma tela cheia com um quadro branco (`SignaturePadPage`): o cliente assina com o dedo (ou mouse). O traço é capturado por ponteiros crus, suavizado com curvas quadráticas pelos pontos médios, e exportado como **PNG recortado** ao retângulo da assinatura (margem de 14 px, ×2,5) sobre fundo branco. A imagem, o nome do signatário e a data/hora ficam no projeto e aparecem no PDF ("Assinado digitalmente em dd/mm/aaaa hh:mm"). É uma assinatura **eletrônica simples** (imagem do traço), sem certificado digital; não substitui uma assinatura com validade jurídica ICP-Brasil.

5.9 Entrega de arquivos (PDF e `.json`)

Ambiente	Comportamento
Navegador / PWA	Download por `Blob` + `<a download>` (`FileIO._browserDownload`)
Shell Android ≥ 3.2.0	`FileIO.deliver` envia o arquivo pela ponte (`saveFile`, em pedaços de base64): **Gerar laudo em PDF** salva em `Downloads/WaveLens`; **Gerar e compartilhar** abre a folha de compartilhamento do Android
Shell Android < 3.2.0	O app mostra "Atualize o app WaveLens Shell (3.2 ou superior)"

O Kotlin usa `MediaStore.Downloads` (Android 10+), sem pedir permissão de armazenamento, e `FileProvider` (`<appId>.fileprovider`, `cache-path shared/`) para o compartilhamento. Nomes de arquivo são sanitizados.

5.10 Estado de validação dos recursos da v8.0

Recurso	Situação
Ferramentas, calibração, materiais, banda, zoom, modos de visualização	Validados no navegador (viewport de celular)
Persistência (recarregar a página), exportar e importar `.json`, upload de planta personalizada	Validados no navegador, incluindo imagem embutida
Laudo em PDF com logo, duas bandas, medição simulada e assinatura	Gerado no navegador e conferido página a página
`saveFile` (Downloads e compartilhamento) no Shell 3.2.0	**Compilado; ainda não validado em aparelho** (sem `adb` disponível)
Medição real (RSSI/ping) vinculada a pontos	Depende do Shell; a leitura real já foi validada na v6.x, o vínculo com pontos ainda não foi testado em aparelho

6. Gestão de Plantas e Mídia

6.1 Modelo de dados

Classe	Campos principais	Papel
`ProjectDef`	`id`, `name`, `subtitle`, `floors`, `isCustom`	Um projeto na biblioteca (casa, prédio, upload)
`FloorDef`	`label`, `plan`	Um pavimento (Térreo, 1º Andar…) da biblioteca
`FloorState`	`label`, `plan`, `widthM`, `calibrated`, `userWalls`	Pavimento **em uso**: planta + escala real + paredes desenhadas (mutável)
`FloorPlanDef`	`id`, `name`, `subtitle`, `imageUrl?`, `assetPath?`, `memoryBytes?`, `aspectRatio`, `defaultWidthM`, `rooms`, `wallSegments`	Uma planta: aparência + física
`RoomDef`	`label`, `rectFrac`	Cômodo do desenho vetorial
`WallSegment`	`a`, `b`, `attenuationDb`, `material?`	Parede para o ray-casting (`material` só nas desenhadas pelo usuário)
`RouterNode`	`id`, `frac`, `floor`, `model`	Roteador posicionado
`MeasurePoint`	`id`, `name`, `frac`, `floor`, `measured?`	Ponto de medição
`Measurement`	`time`, `native`, `rssi`, `linkMbps`, `gwAvgMs`, `netAvgMs`, `*LossPct`, `pointId?` …	Medição de campo
`ReportInfo`	técnico, empresa, cliente, endereço, data, notas, `logo`, `signature`, `signedAt`	Dados do laudo

6.2 Cadeia de resolução da imagem

A planta de fundo é resolvida nesta ordem, com queda automática para a próxima opção em caso de falha:

1. **Memória** (`memoryBytes` → `Image.memory`): plantas enviadas pelo usuário.
2. **Rede** (`imageUrl` → `Image.network`, com indicador de carregamento): arquivo em `raw.githubusercontent.com`.
3. **Asset** (`assetPath` → `Image.asset`): arquivo embutido no app.
4. **Desenho vetorial** (`rooms` → `FloorPlanPainter`): piso texturizado, paredes espessas e silhuetas de móveis.

Isso permite deixar a **vaga** de uma planta reservada: enquanto o PNG não existir, o app mostra o desenho vetorial provisório; ao colocar o arquivo com o nome esperado em `assets/floorplans/`, a imagem passa a ser usada.

6.3 Biblioteca atual

Projeto	Pavimentos	Fonte visual	Situação
Casa Térrea 2 Quartos (`casa_2q`)	1	`casa_2q.png` (736×1105 px)	Imagem limpa, com paredes aproximadas
Casa 2 Pavimentos (Referência)	2	Desenho vetorial próprio	Modelo de referência para casas de dois andares (corte esquemático: sala/escada/sala de TV no térreo, sala com varanda/quarto no 1º andar) — permanentemente vetorial, sem foto associada
Planta 01 — Apartamento 3 Quartos	1	`planta_01.png` (aguardando)	Desenho vetorial provisório

Projeto	Pavimentos	Fonte visual	Situação
Planta 02 — Apartamento Open Space	1	`planta_02.png` (aguardando)	Desenho vetorial provisório
Planta 03 — Apartamento com Terraço	1	`planta_03.png` (aguardando)	Desenho vetorial provisório
Planta 04 — Apartamento 3 Suítes	1	`planta_04.png` (aguardando)	Desenho vetorial provisório (cozinha, sala de jantar, sala de estar, 3 suítes)
Edifício Corporativo	3	`edificio_corporativo.png` (aguardando)	Planta vetorial repetida nos 3 andares (Open Space, Reunião, Diretoria, Copa)

6.4 Como adicionar uma nova planta

1. Confirme que a imagem é **limpa** (sem marca d'água de terceiros e com direito de uso).
2. Copie o arquivo para `netfloor/assets/floorplans/` (o `pubspec.yaml` já inclui a pasta inteira).
3. Em `lib/simulador/dados/biblioteca_plantas.dart`, crie um `FloorPlanDef` com `assetPath`, `imageUrl` (URL `raw.githubusercontent.com` após publicar no `main`) e `aspectRatio = largura / altura` da imagem.
4. Estime as paredes internas em coordenadas fracionárias (0 a 1) e cadastre os `WallSegment` (3,5 / 6 / 10 dB conforme a parede) e a largura real estimada em `defaultWidthM`. Se a planta for nova na biblioteca, inclua-a também em `kBuiltInPlans` (necessário para reabrir projetos salvos e importados).
5. Inclua a planta em um `ProjectDef` na lista `kProjectLibrary`.
6. Publique (seção 7): commit no `main`, build web e deploy no `gh-pages`.

Esse processo já é genérico e está pronto para receber novas plantas a qualquer momento — basta o arquivo de imagem (limpo, sem marca d'água de terceiros) e os 6 passos acima; nenhuma mudança de estrutura é necessária.

6.5 Upload em memória (`Uint8List`)

O botão **Carregar plantas do dispositivo** (biblioteca) e o chip **+ Pavimento** aceitam **uma ou várias** imagens (PNG, JPG, WEBP):

- A leitura usa `FilePicker.pickFiles(type: FileType.custom, allowedExtensions: [...])` e `PlatformFile.readAsBytes()`, obtendo um `Uint8List`. **Não há `dart:io`**, portanto a mesma lógica funciona em Web/PWA, no Shell Android e no desktop. (No `file_picker` 13 o parâmetro antigo `withData: true` não existe mais; os bytes vêm de `readAsBytes()`.)
- As dimensões são obtidas com `instantiateImageCodec`, definindo o `aspectRatio` sem esticar a imagem.
- Cada imagem vira um pavimento, na ordem de seleção (Térreo, 1º Andar...). No botão da biblioteca cria-se um **projeto novo**; no chip **+ Pavimento** os andares são **acrescentados** ao projeto atual.
- A planta é guardada no armazenamento local (store `images`) e vai embutida no `.json` exportado (seção 5.6). Não tem paredes mapeadas: o usuário as desenha com a ferramenta *Parede* e calibra a escala com a *Régua* (seção 5.3).
- No Shell Android, o `<input type="file">` da página é atendido pelo seletor nativo (`setOnShowFileSelector`), com suporte a seleção múltipla a partir do Shell 3.1.0.

6.6 Política de conteúdo e licenças

- Somente imagens **limpas**, próprias ou licenciadas, devem entrar na biblioteca.
- Imagens com marca d'água de terceiros (por exemplo, prévias de bancos de imagens pagos ou de sites de projetos) **não** são incluídas nem têm a marca removida.
- Vagas de plantas cujo arquivo ainda não foi fornecido usam desenho vetorial próprio.
- Quando o usuário envia uma imagem de referência (ex.: um render de terceiros) para pedir uma planta "no estilo" dela, o app **não reproduz a imagem**: um novo `FloorPlanDef` vetorial é desenhado do zero, inspirado apenas na disposição geral dos cômodos (como "Casa 2 Pavimentos" e "Planta 04", v8.1).

6.7 Histórico de alterações

Versão	Data	Alterações
1.0	14/09/2026	Protótipo: duas plantas desenhadas em código (Casa e Apartamento), mapa de calor em 3 faixas (azul/verde/vermelho), toque para adicionar e arraste de roteadores
1.1	15/09/2026	Planta real como fundo; gradiente contínuo "jet" com desfoque; ícone de roteador; anéis pulsantes; correção do repaint do mapa de calor; publicação no GitHub Pages; primeiro APK nativo (release v1.0.0)
2.0	15/09/2026	Catálogo de roteadores (18/23/28 dBm); biblioteca de plantas; fórmula $P_{tx} - 22\log_{10}(d)$; **arquitetura OTA** (site + Shell WebView, release v2.0.0)
4.0	15/09/2026	Plantas ilustradas (piso, móveis, paredes espessas); **ray-casting de paredes**; overlay do calor a ~48 %; `Image.network` com fallback
5.1	16/09/2026	Plantas reais como assets (Casa 2 Quartos com correção de espelhamento, Casa 3 Quartos e Apartamento 2 Quartos, este com marca d'água de terceiros aceita pelo usuário na época)
6.0	18/09/2026	Upload de plantas em memória; **WaveLens Diagnostic** (espectro, sinal, latência); ponte JS + Kotlin; Shell 3.0.0; assets movidos para `assets/floorplans/`
6.1	18/09/2026	Destaque e sublinhado da rede conectada e da rede de mesmo SSID em outra banda; saúde do canal desconsidera a própria rede
7.0	19/09/2026	**Projetos com vários pavimentos (2.5D)**, distância 3D e perda de laje de 15 dB; posições dos roteadores em frações; upload em lote; novas plantas limpas; cartão "Rede conectada"; Shell 3.1.0
7.0.1	19/09/2026	**Limpeza da biblioteca:** remoção do projeto Sobrado (térreo e 1º andar) e do arquivo `sobrado_1andar.png`; vagas `planta_01..03` com desenho provisório
8.0	21/09/2026	**WaveLens Enterprise:** laudo de vistoria em PDF com logo, mapas por banda, tabelas previsto × medido e **assinatura digital**; `InteractiveViewer` (zoom/pan); **calibração de escala por régua** (o cálculo passa a ser em metros reais); **materiais de parede** e ferramenta de desenho; **2.4 / 5 / 6 GHz**; pontos e **medição de campo** (RSSI, PHY, ping duplo, perda); modos **Apresentação** e **Diagnóstico**; **persistência offline** (IndexedDB) e exportar/importar `.json`; Shell 3.2.0 (`saveFile`, compartilhamento, seletor `.json`); a constante `kPixelsPerMeter` deixa de existir
8.1	25/09/2026	**Biblioteca:** novo projeto "Casa 2 Pavimentos (Referência)" (2 andares, desenho vetorial próprio, modelo para simulações em casas de dois pisos) e "Planta 04 — Apartamento 3 Suítes"; heurísticas de mobília do desenho vetorial ganham mesa de jantar (`jantar`) e escada (`escada`). **Catálogo:** novo roteador **ZTE E2320 / E2620 (ZXHN, Wi-Fi 6)**, 20 dBm (acima do Huawei AX3), com ícone próprio (corpo branco, duas antenas externas)
8.2	26/09/2026	**Rebranding:** o produto passa a se chamar **WaveLens** (antes NetFloor) — título, cabeçalhos, banners, metadados do laudo em PDF e `android:label` do APK atualizados; identificadores técnicos internos (repositórios GitHub, `applicationId` Android, ponte `NetFloorNative`, canal `netfloor/diag`, formato `netfloor-project`) mantidos por compatibilidade (seção 1.5). **Ícone novo:** desenho próprio (wifi + roteador + globo, azul), com `flutter_launcher_icons` configurado no Shell e script de redimensionamento para os ícones do site/PWA (seção 2.6). Shell 3.3.0 (novo `android:label`, pasta de downloads passa a ser `Downloads/WaveLens`)
8.2.1	18/10/2026	**Organização do código:** o `main.dart` (6,7 mil linhas) foi dividido em 30 módulos (`app/`, `core/`, `simulador/`, `diagnostico/`, `infra/`, `laudo/`) com `part` / `part of`, cada um com cabeçalho "ETAPA n" para explicar o app etapa por etapa (seção 2.3 e `GUIA_DO_CODIGO.md`); nenhuma linha de código mudou (verificado). Pequeno ajuste do rebranding: o projeto exportado passa a se chamar `<nome>.wavelens.json` (a importação continua aceitando qualquer `.json`)

Remoções relacionadas a marca d'água e conteúdo.

Item	Situação
`Apartamento 2 quartos.jpg` (marca `@opedreiro`)	Removido do código, da biblioteca e do site na v7.0
Antigas Casa 2 Quartos e Casa 3 Quartos	Substituídas pela nova biblioteca na v7.0
Sobrados (`sobrado_1andar.png` e desenho do térreo)	Removidos na v7.0.1

Item	Situação
Imagens com marca de terceiros enviadas depois (`montesuacasa.com.br`, `depositphotos`)	**Não incluídas** no projeto

> **Observação importante:** a remoção é da **biblioteca, do código e do site publicado**. Os arquivos antigos continuam recuperáveis no **histórico de commits do Git** do repositório. Apagá-los de vez exige reescrever o histórico (`git filter-repo` e `push --force`), o que só deve ser feito com pedido explícito.

7. Guia de Instalação, Build e CI/CD

7.1 Ambiente de desenvolvimento (Windows 11)

Componente	Local / versão
Flutter SDK	`C:\flutter` — 3.47.4 stable (Dart 3.13.3); `C:\flutter\bin` no PATH
JDK	Eclipse Temurin 17.0.20 — `JAVA_HOME = C:\Program Files\Eclipse Adoptium\jdk-17.0.20.101-hotspot`
Android SDK	`C:\Android\sdk` — `ANDROID_HOME` / `ANDROID_SDK_ROOT`; plataformas 35 e 36; build-tools 28.0.3, 35.0.0 e 36.0.0; `platform-tools` e `cmdline-tools` no PATH
Git e GitHub CLI	`git` e `gh` autenticado na conta do projeto
Navegador	Chrome/Edge para desenvolvimento web

7.2 Dependências (`pubspec.yaml`)

App web (`netfloor`)

```yaml
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.6
  file_picker: ^13.1.0   # upload de plantas/logo/.json em memória (bytes), sem dart:io
  fl_chart: ^1.2.0       # gráficos de linha (sinal e latência)
  pdf: ^3.13.1           # laudo de vistoria em PDF (100 % Dart)
  image: ^4.10.1         # codificação JPEG dos mapas de calor do laudo
  web: ^1.1.1            # download de arquivos (Blob) e visibilitychange
  sembast: ^3.8.11       # banco NoSQL local
  sembast_web: ^2.4.6    # ... sobre IndexedDB (offline-first)

dev_dependencies:
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
  assets:
    - assets/floorplans/
    - assets/icon/        # icon.png (fonte dos ícones web/PWA, seção 2.6)
    - assets/fonts/       # Roboto (Regular, Bold, Italic) para o PDF
```

Shell Android (`netfloor_shell`)

```yaml
version: 3.3.0+6
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  webview_flutter: ^4.10.0
  webview_flutter_android: ^4.14.1
  permission_handler: 12.0.1   # a 13.x exige compileSdk 37 (não suportado pelo Gradle atual)
  file_picker: ^13.1.0         # seletor de arquivos para o <input type="file">

dev_dependencies:
  flutter_launcher_icons: ^0.14.4   # gera ic_launcher.png em mipmap-* a partir de assets/icon/icon.png
```

7.3 Permissões do Android (`AndroidManifest.xml` do Shell)

Permissão	Uso
`INTERNET`	Carregar o app web e as imagens

Permissão	Uso
`ACCESS_NETWORK_STATE`	Estado da conectividade
`ACCESS_WIFI_STATE`	Ler `WifiInfo`, `DhcpInfo` e resultados de varredura
`CHANGE_WIFI_STATE`	Solicitar `startScan()`
`ACCESS_FINE_LOCATION`, `ACCESS_COARSE_LOCATION`	Exigidas pelo Android para varredura Wi-Fi e leitura de SSID/BSSID
`NEARBY_WIFI_DEVICES`	Permissão de dispositivos Wi-Fi próximos (Android 13+)

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.NEARBY_WIFI_DEVICES"/>
```

Além das permissões, o manifesto do Shell 3.2.0 declara um `FileProvider` (`androidx.core.content.FileProvider`, autoridade `${applicationId}.fileprovider`, caminhos em `res/xml/file_paths.xml`: `cache-path shared/`) usado para compartilhar o laudo. Salvar em Downloads não exige permissão (usa `MediaStore` no Android 10+).

Identificação do Shell: `applicationId = com.netfloor.netfloor_shell`, rótulo **WaveLens**, `versionName 3.2.0`, `versionCode 5`. O APK é assinado com a **chave de debug** do Flutter (adequado a uso pessoal; a Play Store exigiria uma keystore própria).

7.4 Desenvolvimento local

```
cd "C:\Users\Roger\Desktop\Nova pasta\netfloor"
flutter pub get
run_web.bat         :: flutter run -d web-server --web-port 8090 --web-hostname 0.0.0.0
```

Acesse `http://localhost:8090`. Na primeira compilação a tela pode ficar preta por ~1 minuto; recarregue a página quando o terminal indicar que o app está sendo servido. Com `--web-hostname 0.0.0.0`, outros aparelhos da mesma rede acessam pelo IP do computador (pode ser necessário liberar a porta no Firewall do Windows).

7.5 Build web de produção

```
flutter build web --release --base-href "/netfloor/" --pwa-strategy=none
```

Opção	Motivo
`--base-href "/netfloor/"`	O site é servido em um subcaminho do GitHub Pages
`--pwa-strategy=none`	Desliga o cache de service worker: cada abertura busca a versão mais recente (base do OTA). Como consequência, o app precisa de internet para abrir. A opção está marcada como **obsoleta** no Flutter 3.47 (funciona, mas será removida em versão futura)

7.6 Pipeline de implantação contínua (OTA) via GitHub Pages

Não há GitHub Actions: o pipeline é manual e reproduzível, sempre nesta ordem.

1. **Commit e push do código** no `main` (assim as imagens ficam disponíveis em `raw.githubusercontent.com` antes do site que as referencia).
2. **Build web** (seção 7.5).
3. **Publicar** o conteúdo de `build/web` no branch `gh-pages` (histórico descartável, sobrescrito):

```
cd netfloor/build/web
rm -rf .git && git init -q && git checkout -q -b gh-pages
git add -A && git commit -q -m "Deploy: WaveLens vX.Y"
git remote add origin https://github.com/Rogerdev5690/netfloor.git
git push -f origin gh-pages
```

4. **Aguardar** o GitHub Pages (`gh api repos/Rogerdev5690/netfloor/pages/builds/latest --jq .status` até `built`) e conferir o site.
5. O aplicativo instalado carrega a nova versão **na próxima abertura**, sem reinstalar o APK.

7.7 Build e versionamento do APK Shell

```
cd "C:\Users\Roger\Desktop\Nova pasta\netfloor_shell"
:: pubspec.yaml -> version: 3.3.0+6  (e kShellVersion em lib/main.dart)
:: Trocou o ícone/label? Rode antes: dart run flutter_launcher_icons (seção 2.6)
flutter build apk --release
gh release create v3.3.0 build/app/outputs/flutter-apk/app-release.apk#WaveLens-v3.3.0-shell.apk ^
  --repo Rogerdev5690/netfloor --title "WaveLens Shell v3.3.0" --notes "..."
```

Release	Conteúdo
v1.0.0	Primeiro APK nativo em Flutter (histórico; obsoleto)
v2.0.0	Primeiro Shell WebView (arquitetura OTA)
v3.0.0	Shell com diagnóstico nativo (ponte JS, Wi-Fi, permissões, seletor de arquivos)
v3.1.0	Seleção múltipla de imagens no seletor de arquivos
v3.2.0	Entrega de arquivos (`saveFile`: Downloads e compartilhamento), seletor de `.json` para importar projetos
v3.3.0	Rebranding: `android:label` = WaveLens, novo ícone do launcher, pasta `Downloads/WaveLens` (**atual**)

Regra de versão do Shell: incrementar o `versionCode` (o número após `+`) a cada APK novo; mudar o número maior quando houver novos recursos nativos. O APK atualiza o anterior por cima (mesmo `applicationId` e mesma chave de assinatura).

Instalação no celular: baixar o APK do release e instalar (o Android pede para permitir "instalar apps desconhecidos"), ou, com Depuração USB ativa, `adb install -r app-release.apk`. Em aparelhos Xiaomi pode ser necessário habilitar também "Instalar via USB".

7.8 Checklist de release

- [ ] `flutter analyze` sem erros no app web e no Shell.
- [ ] Teste no navegador: simulador, troca de andares, upload, ferramentas (parede, régua, ponto), troca de banda e de modo de visualização, persistência (recarregar a página), exportar/importar, geração do laudo e as três abas do diagnóstico em simulação.
- [ ] Commit e push no `main`; build web; deploy no `gh-pages`; site verificado.
- [ ] Se houve mudança nativa: novo APK, `versionCode` incrementado, release publicado.
- [ ] **Atualizar `docs/DOCUMENTACAO.md` e regenerar o PDF** (seção 9).

7.9 Solução de problemas

Sintoma	Causa	Solução
"Building with plugins requires symlink support"	Plugins com pasta `windows/` exigem o Modo Desenvolvedor do Windows	O projeto web não usa a plataforma Windows; a pasta `windows/` foi removida
Gradle: `Failed to find target with hash string 'android-37'`	`permission_handler` 13 exige compileSdk 37	Usar `permission_handler` 12.0.1

Sintoma	Causa	Solução
Tela preta na 1ª abertura do servidor local	Compilação inicial do Flutter Web	Aguardar e recarregar
Espectro vazio no Android	Localização negada ou GPS desligado; limite de varreduras	Conceder permissão, ligar o GPS e tocar em **Escanear**
`adb devices` não lista o celular	Depuração USB desligada ou modo USB incorreto	Ativar Depuração USB (e "Instalar via USB" em Xiaomi) e autorizar o computador
Imagem da planta não aparece	URL do `raw.githubusercontent.com` ainda inexistente	Fazer push no `main`; o app usa o asset embutido como reserva
`withData` não existe no `file_picker`	API mudou na versão 13	Usar `readAsBytes()` do `PlatformFile`
Não consigo arrastar roteadores dentro do `InteractiveViewer`	O reconhecedor de escala vence o `onPanUpdate` do filho	Tratar toques com `Listener` (ponteiros crus), como no simulador
`fontSize != null …` `fontSizeFactor` ao trocar de tema	`TextTheme.apply(fontSizeFactor)` não aceita estilos sem tamanho	Escalar o texto por `MediaQuery.textScaler`
Texto ilegível em cartões claros no tema escuro	Cores de fundo fixas (âmbar, verde, laranja) com texto herdado do tema	Fixar `Colors.black87` nesses cartões
PDF muito grande (> 5 MB)	Imagem bruta RGBA incorporada	Codificar o mapa em JPEG antes de incorporar
"Atualize o app WaveLens Shell" ao gerar o laudo no celular	Shell anterior à 3.2.0 não tem `saveFile`	Instalar o APK 3.2.0

7.10 Segurança e privacidade

- Nenhum dado do usuário é enviado a servidores: projetos, plantas enviadas, laudos, logos e assinaturas ficam no IndexedDB do aparelho (ou no arquivo que o usuário exportar); leituras Wi-Fi são exibidas localmente. Limpar os dados do site/aplicativo apaga os projetos salvos: exporte o `.json` para ter cópia.
- O Shell só carrega o domínio do projeto; a ponte nativa não é acessível a outros sites.
- O ping usa apenas os destinos exibidos (gateway local e 8.8.8.8).
- As permissões de localização são usadas exclusivamente porque o Android as exige para varredura Wi-Fi.

8. Limitações Conhecidas e Pendências

Tema	Descrição
Plantas provisórias	`planta_01..03` e `edificio_corporativo` aguardam imagens limpas; usam desenho vetorial
Proporção fixa por planta	O `aspectRatio` de cada planta é definido no código; ao trocar um PNG por outro de proporção diferente, o valor deve ser ajustado
Paredes aproximadas	As da biblioteca são visuais; as das plantas enviadas dependem do usuário desenhá-las
Modelo de RF simplificado	Ver seção 3.9; o nível simulado não é o RSSI real
Uso offline	Os **dados** ficam salvos offline, mas com `--pwa-strategy=none` o app ainda precisa de internet para **abrir** (o service worker foi desligado para garantir o OTA)
Assinatura eletrônica simples	É a imagem do traço, sem certificado digital
`saveFile` no Shell 3.2.0	Compilado; ainda não validado em aparelho
Medições no navegador	Simuladas (marcadas como tal no laudo); medição real exige o Shell
Assinatura do APK	Chave de debug: não serve para Play Store
Testes automatizados	Inexistentes; validação é manual
Histórico do Git	Arquivos removidos (por exemplo, com marca d'água) continuam nos commits antigos
Pasta `netfloor/android/`	Legado do APK nativo v1.0.0; o projeto atual depende de `dart:js_interop` e só compila para Web
Seleção múltipla no Shell 3.1.0	Compilada, ainda não validada em aparelho

9. Manutenção desta Documentação (Regra de Projeto)

Esta é a **versão base** da documentação. A cada nova atualização do aplicativo:

1. Implementar a alteração pedida.
2. Atualizar a(s) seção(ões) correspondente(s) de `docs/DOCUMENTACAO.md`, incluindo o **histórico de alterações** (seção 6.7) e, se necessário, a tabela de releases (7.7).
3. Ajustar as três marcas no topo do arquivo (`doc-version`, `doc-revision`, `doc-date`).
4. Regenerar o PDF e entregá-lo ao usuário:

```
python "C:\Users\Roger\Desktop\Nova pasta\netfloor\docs\build_pdf.py"
```

O script converte o Markdown em HTML (com fórmulas em MathML), imprime em PDF pelo Chrome em modo headless, calcula os números de página do sumário, acrescenta cabeçalho/rodapé com numeração e cria os marcadores (bookmarks) do PDF. Dependências Python: `markdown`, `latex2mathml`, `pygments`, `pypdf`, `reportlab`.

Apêndice A — Glossário

Termo	Significado
dBm	Decibel-miliwatt: unidade logarítmica de potência; valores mais próximos de 0 são sinais mais fortes (−50 dBm é melhor que −80 dBm)
RSSI	Received Signal Strength Indicator: nível de sinal recebido pelo aparelho
SSID / BSSID	Nome da rede / identificador (MAC) do ponto de acesso específico
PHY (velocidade)	Taxa de dados negociada na camada física entre aparelho e roteador (não é a velocidade da Internet)
Canal / largura	Faixa de frequência usada por uma rede (20, 40, 80 ou 160 MHz)
Ray-casting	Traçado de um raio entre dois pontos para contar obstáculos cruzados
2.5D	Simulação em planos empilhados (andares) sem modelo 3D completo
Laje	Estrutura horizontal de concreto entre pavimentos
PWA	Progressive Web App: aplicativo web instalável
OTA	Over-the-air: atualização sem reinstalar o aplicativo
WebView	Componente do Android que exibe páginas web dentro de um app
Mesh	Rede com vários nós que cooperam para ampliar a cobertura
Calibração (régua)	Ajuste da escala metros/pixel a partir de uma distância real conhecida
IndexedDB	Banco de dados do navegador/WebView que guarda os projetos offline
Laudo de vistoria	Documento PDF entregue ao cliente com mapas, medições e assinatura
Gateway	Roteador local da rede (destino do ping "local")

Apêndice B — Referência rápida de constantes

Constante	Valor	Onde
`FloorPlanDef.defaultWidthM`	8 m (padrão)	Largura estimada da planta até calibrar (12/12/10/20 nas plantas 01/02/03/edifício; 10 em uploads)
`kFloorHeightM`	3,0 m	Distância vertical entre andares
`kSlabLossDb`	15 dB	Perda por laje (todas as bandas)
`kStrongDbm` / `kWeakDbm`	12 / −10 dBm	Limiares Forte / Ruim
`kRfBands`	2.4: (0, 22, 1,00) · 5: (−6,4, 24, 1,35) · 6: (−7,9, 25, 1,50)	(Δref dB, k, fator de paredes)
Materiais de parede	Gesso 2 · Cerâmico 4 · Vidro 5 · Concreto 12–15 (13,5) dB	`kWallMaterials`
Zoom máximo	12×	`InteractiveViewer.maxScale`
Autosave	0,8 s	Após a última alteração
Largura do mapa no PDF	1000 px, JPEG q88	`renderFloorRaster`
Pedaço de arquivo na ponte	256 KB (base64)	`FileIO.deliver` → `saveFile`
`_kDbmFloor` / `_kDbmCeil`	−30 / +26 dBm	Escala de cor do mapa de calor
`kDefaultHeatOpacity`	0,48	Opacidade padrão do calor
`cell` (grade)	8 px (proporcional no PDF)	Amostragem do mapa de calor
`kMaxSamples`	60	Amostras exibidas nos gráficos de linha
`kDnsHost`	`8.8.8.8`	Alvo do ping de Internet
Intervalo de varredura	30 s	Aba Espectro
Intervalo de sinal e ping	1 s	Abas Sinal e Latência